Homework 1: Language Models from Scratch

CMPE 258 · Character-level models on Tiny Shakespeare, trained with nanolm · commit 8418fc1bae · RTX 5070 Laptop GPU, PyTorch 2.11.0+cu128, bfloat16
Code, configs, results and figures: github.com/nikhil-ghind/cmpe258-hw1-language-models

1. Question and hypothesis

This homework is about two things: understanding what the loss numbers actually mean, and seeing how recurrent networks and Transformers behave differently in practice. Going in, I had three expectations. **(H1)** The vanilla RNN would lose badly to the GRU and LSTM, because gating is supposed to fix vanishing gradients. **(H2)** The bigger LSTM would overfit, since Tiny Shakespeare is only about 1 MB of text. **(H3)** The Transformer would get more out of longer context than the LSTM, because attention can look straight back at distant characters.

Only one of these held up cleanly. H2 was right. H1 was only partly right: most of the RNN's gap turned out to be about its learning rate and size, not its lack of gates. H3 was not supported at all.

2. Setup

All models are trained on Tiny Shakespeare, with the last 10% held out for validation, using a character tokenizer (65 symbols) unless I say otherwise. To keep comparisons fair, every training run in Sections 3.2–3.4 sees exactly the same 16.384 million tokens: 2000 steps of 8192 tokens each, which is about 16 passes over the corpus. Using the usual estimate of 6 × parameters × tokens, that works out to between 0.03 and 0.42 PFLOP per run depending on model size (Table 2).

I used seeds 1337 and 42 for every configuration, and added seed 7 for the context-length study; compared configurations always share the same seeds. The metric throughout is final validation bits per byte (bpb) on the full validation split. To decide what counts as a real difference, I used the seed-to-seed noise floor reported by the analysis tool (0.006 bpb for the model comparison, 0.0069 for the context study) and called any gap smaller than about twice that "not resolved". The p-values come from paired tests on only two or three seeds, so I treat them as weak evidence at best.

In total I ran 82 training runs, which took 61 minutes of GPU time including evaluation. Every experiment is defined by a sweep file in the repository; I did not modify the nanolm library itself.

3. Results

3.1 Metrics by hand

Quantity (modern Transformer, seed 1337)	From metrics file	My own pass	Logged
Loss (nats/token)	1.457936	1.457937	1.457936
Perplexity = e^{loss}	4.29708	4.29708	4.29708
Bits/token = loss / ln 2	2.103358	2.103358	—
bpb = bits/token × tokens/bytes	2.103358	2.103358	2.103358

Table 1a. The validation split has 111,360 tokens and 111,360 bytes (435 windows of 256 characters, one byte each), so bits per token and bits per byte are the same number here. "My own pass" re-scores the raw validation text with a separate loop, without using the library's evaluation code.

Tokenizer	V	ln V	Loss, no update	− ln V	Loss, 1 step	− ln V	bpb, no update	Uniform bpb
Characters	65	4.174	4.224	+0.050	4.168	-0.006	6.094	6.022
Bytes	256	5.545	5.510	-0.035	5.442	-0.103	7.950	8.000
GPT-2 BPE	50,257	10.825	10.825	+0.000	10.796	-0.029	5.050	5.050

Table 1b. Loss of an untrained model, averaged over two seeds. "1 step" is the assignment's suggestion of training for a single step; "no update" is the same run with the learning rate set to zero, so the weights never change.

Everything lines up. Recomputing loss, perplexity, bits per token and bpb from the saved totals gives exactly the logged values, and my independent pass over the text differs by only 4×10^{-7} nats, which is just float32 rounding. For untrained models, the loss sits within 0.05 nats of ln V for all three tokenizers, which is what you'd expect from a model that is guessing roughly uniformly. One detail surprised me at first: after a single training step the loss can already dip *below* ln V, because even one warmup step teaches the model a little about which characters are common. The last two columns show why bpb is the fairer unit. Per-token loss ranges from 4.2 to 10.8 across tokenizers, but once you convert to bits per byte the uniform-guess baselines are 6.02, 8.00 and 5.05, which are directly comparable.

3.2 RNN vs. LSTM vs. GRU vs. Transformer

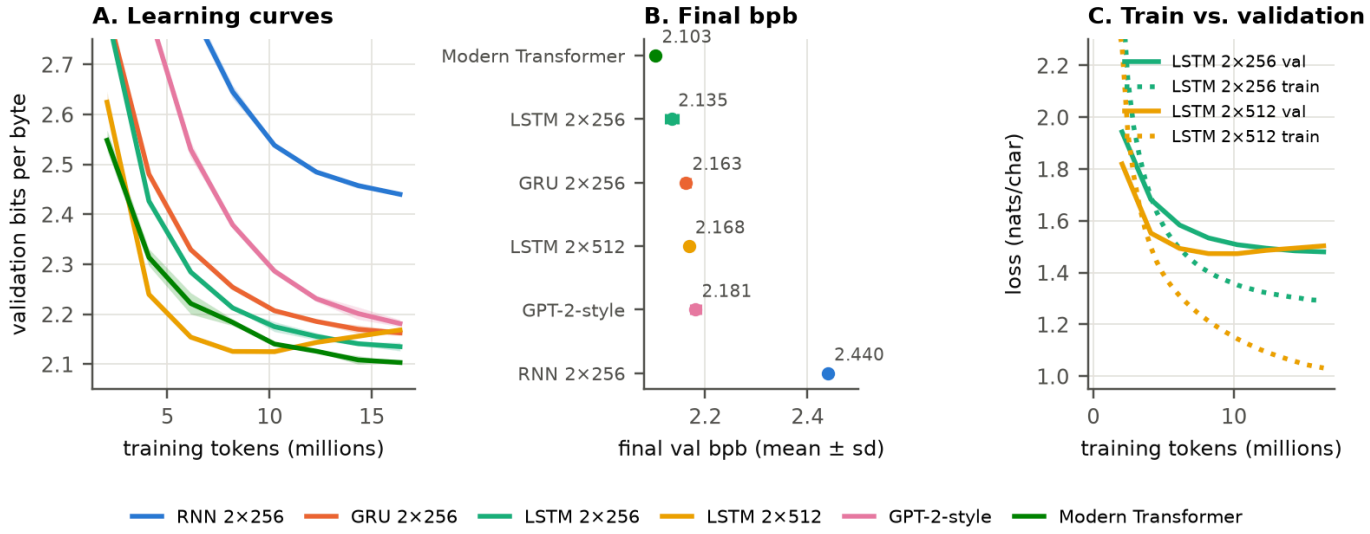


Figure 1. The course's standard model-comparison sweep, with each model at its default learning rate (2 seeds; shaded band shows the range). A: validation bpb as training goes on. B: final bpb, mean ± standard deviation. C: training loss (dotted, with dropout on) against validation loss for the two LSTMs.

Model	Params	PFLOP	Val bpb (mean ± sd)	Δ vs. GPT-2 style	Paired p	Train s	Best bpb after LR tuning
Transformer, modern	3.16M	0.31	**2.1033 ± 0.0001**	-0.078	0.058	47	2.103 (lr 1e-3)
LSTM 2×256	1.07M	0.11	**2.1350 ± 0.0119**	-0.046	0.205	19	2.101 (lr 4e-3)
GRU 2×256	0.81M	0.08	**2.1625 ± 0.0071**	-0.019	0.066	18	2.113 (lr 4e-3)
LSTM 2×512	4.24M	0.42	**2.1683 ± 0.0049**	-0.013	0.434	36	not tuned
Transformer, GPT-2 style	3.44M	0.34	**2.1813 ± 0.0099**	baseline	—	33	not tuned
Vanilla RNN 2×256	0.28M	0.03	**2.4400 ± 0.0005**	+0.259	0.018	6	2.245 (lr 8e-3)

Table 2. Every model gets the same training budget. The default learning rates are 2e-3 for the recurrent models and 1e-3 for the Transformers; the last column is the best result from my own learning-rate search (Figure 2).

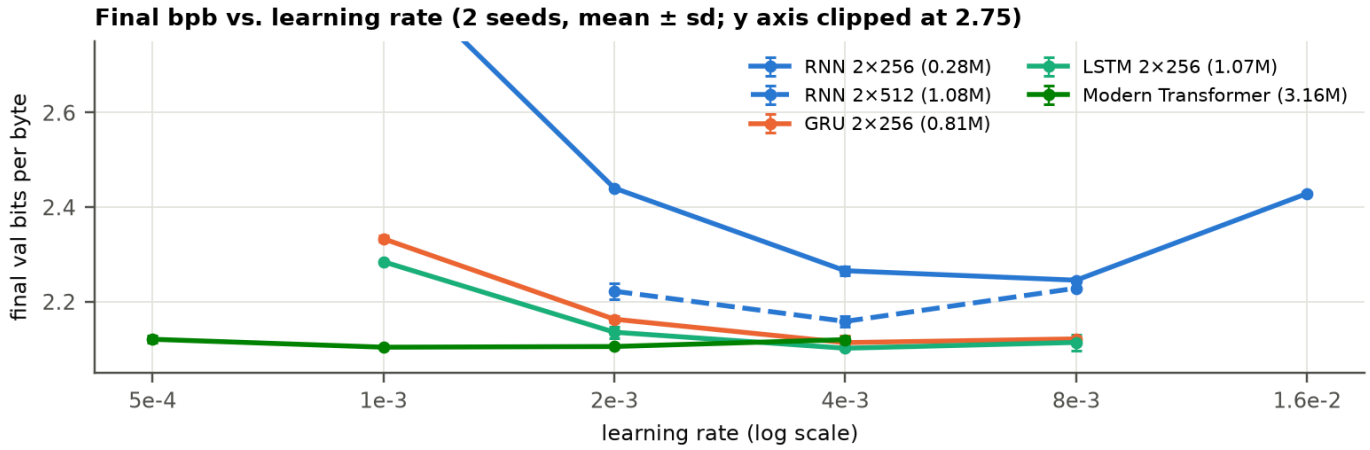


Figure 2. My learning-rate search, which goes beyond the standard sweep. I made sure each model's best value had a worse value on both sides. Best rates: 1e-3 for the Transformer, 4e-3 for the LSTM and GRU, and 8e-3 for the RNN at width 256 (4e-3 at width 512).

The headline ranking turns out to depend heavily on learning rate. With the default settings, the modern Transformer beats the LSTM by 0.032 bpb, which looks like a clear win. But once the LSTM gets its own best learning rate, it reaches 2.1011 ± 0.0002, essentially level with the Transformer's 2.1033 ± 0.0001. The difference (-0.0022) is below the noise floor, so I call it **not resolved**. The paired p-value of 0.02 looks impressive, but it only reflects two seeds that happened to agree closely, so I don't read anything into it.

3.3 Context length

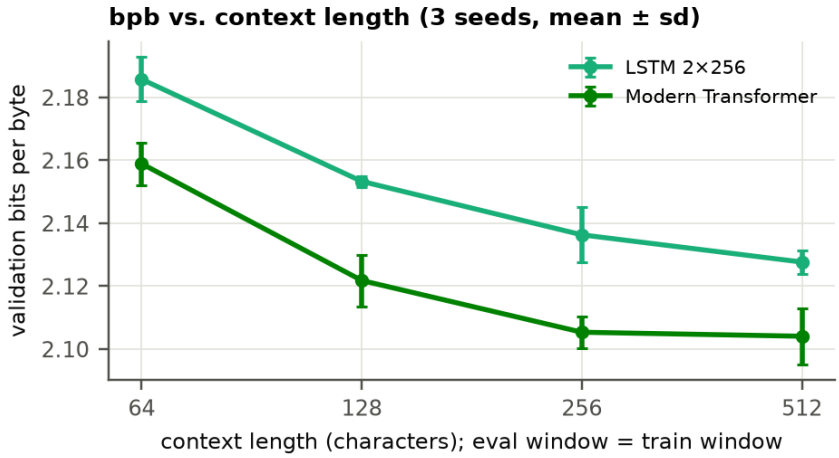


Figure 3. Validation bpb against context length for the LSTM and the modern Transformer. Each model is evaluated on windows the same length it was trained on.

Model	T	Val bpb (W = T)	Change from previous T	
LSTM	64	**2.1857 ± 0.0071**	—	—
LSTM	128	**2.1533 ± 0.0017**	-0.0324	resolved
LSTM	256	**2.1363 ± 0.0087**	-0.0170	resolved
LSTM	512	**2.1276 ± 0.0037**	-0.0087	not resolved
Transformer	64	**2.1589 ± 0.0068**	—	—
Transformer	128	**2.1218 ± 0.0082**	-0.0371	resolved
Transformer	256	**2.1053 ± 0.0050**	-0.0164	resolved
Transformer	512	**2.1040 ± 0.0088**	-0.0013	not resolved

Table 3a. Three seeds each. Every run uses the same 16.4M tokens and 2000 steps; I changed the batch size so the number of tokens per step stays fixed. A change counts as resolved if it is at least 0.0138 bpb, twice the noise floor.

Where does each model stop benefiting? Both models improve clearly from 64 to 128 and from 128 to 256 characters. The step from 256 to 512 is too small to resolve for either one: the LSTM gains 0.0087 bpb and the Transformer only 0.0013, both under the 0.0138 threshold. So both stop benefiting at around 256 characters. The Transformer flattens out a little more sharply, gaining almost nothing past 256, while the LSTM is still drifting down slightly at 512, though not by enough to call it real.

Even the gains below 256 are partly an accounting effect rather than the model using long-range information. Every evaluation window starts with no context, so the first few characters are hard to predict. A longer window spreads that fixed cost over more characters, which lowers the average bpb even if the model never uses anything far back. **On this corpus, neither architecture gets measurably better from context beyond about 256 characters.**

3.4 Attention analysis

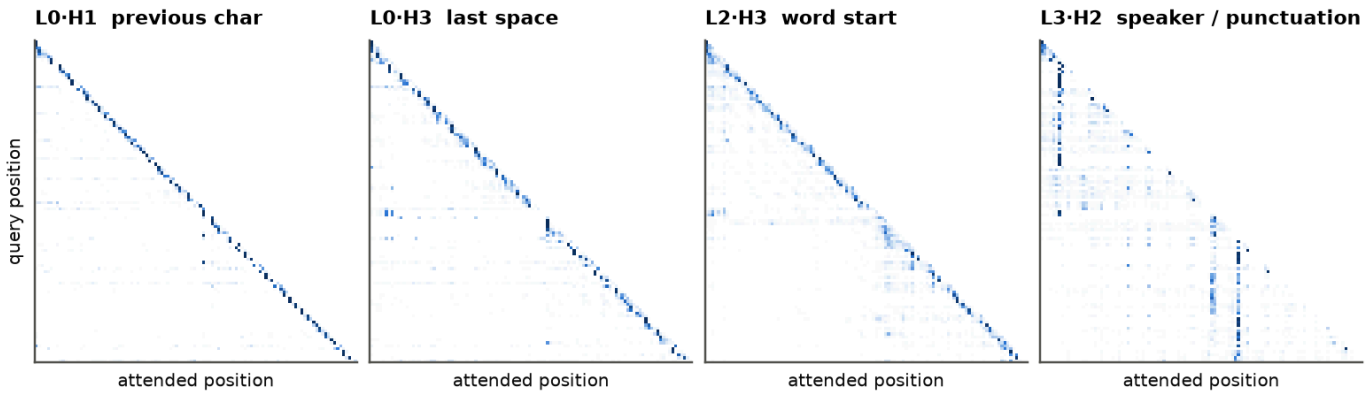


Figure 4. Four attention heads from the modern Transformer (seed 1337) on the dialogue text, which begins "ROMEO: But soft! what light…"; darker means more attention. All 16 heads for all three texts are in Appendix B, made with the course's attention visualizer.

Head type	Head	Dialogue	Verse	Modern prose	Validation mean ± SE	Untrained
Previous char	s1337 L0·H1	0.431	0.566	0.540	0.464 ± 0.003	0.020
	s42 L0·H3	0.628	0.769	0.803	0.671 ± 0.003	0.020
Last space (word boundary)	s1337 L0·H3	0.290	0.402	0.305	0.349 ± 0.005	0.016
	s42 L0·H0	0.204	0.275	0.212	0.227 ± 0.004	0.016
Word start	s1337 L2·H3	0.300	0.217	0.232	0.281 ± 0.005	0.019
	s42 L2·H3	0.371	0.217	0.287	0.304 ± 0.006	0.019
Speaker tag / punctuation	s1337 L3·H2	0.514	0.063	0.040	0.469 ± 0.004	0.091
	s42 L1·H0	0.520	0.067	0.013	0.478 ± 0.004	0.088

Table 4. Average attention weight on the position each head type is supposed to look at (as in §1.6.2 of the tutorial). I picked the heads using 80 validation windows of 256 characters, then scored them separately on the three texts. The word-start score only counts characters at least two letters into a word, so it can't be confused with the previous-character head. The punctuation score is the total weight on colons, periods, exclamation and question marks, semicolons, commas and line breaks. For reference, uniform attention would score about 0.035 for the previous character and 0.017 for word start on the verse text.

I found four kinds of heads, and each one shows up in both seeds (the layer and head numbers differ between seeds, which is expected since nothing ties them together):

- **Previous character:** puts 0.46–0.67 of its attention on the character right before, compared with 0.020 for an untrained model.
- **Last space:** looks back to the most recent space, i.e. the start of the current word (0.23–0.35 vs. 0.016 untrained).
- **Word start:** attends to the first letter of the current word (0.28–0.30 vs. 0.019 untrained).
- **Speaker tag / punctuation:** 0.47–0.48 vs. about 0.09 untrained. This one is clearly tied to the text type. On the dialogue it scores 0.51–0.52, but only 0.01–0.07 on the verse and modern prose, which don't have speaker names followed by a colon and a line break. It also looks back further than any other head, about 25 characters on average.

4. Analysis

Why does the vanilla RNN fall so far behind? At the default settings it trails the LSTM by +0.305 bpb (p = 0.017), and it is clearly underfitting rather than overfitting: its training loss is 1.62 against 1.69 on validation, while the LSTM gets its training loss down to 1.29. To find out where the gap comes from, I closed it one step at a time against the tuned LSTM. The total gap is 0.339 bpb. Raising the RNN's learning rate from 2e-3 to 8e-3 closes **0.195** of it. Making it wider (0.28M to 1.08M parameters, the same size as the LSTM) closes another **0.087**. What's left, **0.057**, is the actual benefit of gating: an RNN and an LSTM of equal size, both well tuned, score 2.158 and 2.101, a gap about 4.7 times the seed spread. So gates do help, but they only account for about 17% of the gap. The RNN is also far more sensitive to the learning rate (Figure 2). Across rates from 1e-3 to 8e-3 it ranges from 2.25 to 2.92 bpb, while the LSTM stays between 2.10 and 2.28.

I also expected to see the classic vanishing-gradient picture, and I didn't (Figure 5). In the trained models, the gradient from the last character's loss back to earlier inputs shrinks at almost the same rate for the RNN, GRU and LSTM: 0.094, 0.119 and 0.175 at 8 characters back, and 1.2e-03, 6.0e-04 and 1.2e-03 at 64 characters back. I think this is a genuine negative result rather than a bug, because the same measurement gives the Transformer a flat profile that reaches far back, exactly as it should. It doesn't rule out gates helping *during* training, though, since I only measured the models after training finished.

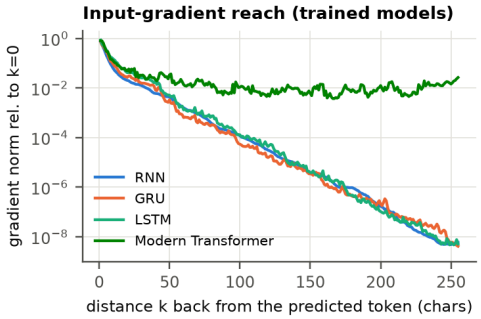


Figure 5. How strongly the last character's loss depends on the input k characters earlier (average gradient size, relative to k = 0; 48 windows × 2 seeds). The Transformer stays around 10^{-2} all the way back to k = 255.

Is the larger LSTM better or worse? Worse, at this training budget. It finishes at 2.168 ± 0.005 against 2.135 ± 0.012 for the smaller one, a gap of +0.033 that is 5.5 times the noise floor (p = 0.09 with two seeds). Figure 1 shows what's going on. Early in training the big LSTM is actually the best model of all, bottoming out at 2.126 and 2.121 bpb after 8–10M tokens. After that its validation loss starts *climbing*, by 0.044–0.046 bpb, even though its training loss keeps falling, down to 1.02. By the end its train/validation gap is 0.49 nats, compared with 0.20 for the smaller LSTM. That's textbook overfitting: with four times the parameters and only 1 MB of text, it starts memorizing. If I had stopped it at its best point (2.123), it would beat the small LSTM at the same learning rate (2.135) by 0.012, which is right at the edge of what I can resolve, and it would still lose to the tuned small LSTM (2.101). So the bigger model only comes out ahead with early stopping, and even then the advantage isn't clearly real.

Which architecture uses longer context more effectively? Measured in bpb, neither one clearly does. Going from 64 to 512 characters of context, the LSTM improves by 0.058 bpb and the Transformer by 0.055, almost the same. The Transformer is ahead by 0.024–0.032 at every length, but that is the same lead it had at default learning rates in Section 3.2, not something that grows with context. Where they really differ is in *how* they handle context. The Transformer can reach much further back: at 128 characters its gradient is 6.9e-03 versus

2.2e-05 for the LSTM, roughly 300 times larger. But that extra reach doesn't turn into better predictions on this corpus, and it only works at the length the model was trained on. The Transformer trained on 64-character windows scores 2.16 bpb on 64-character windows but falls apart at 512 (4.40). The LSTM has no such problem: trained on 64 characters, it scores 2.127 on 512-character windows, matching the LSTM actually trained at 512 (2.128). So the Transformer has the longer reach, the LSTM is more flexible, and neither gets a measurable benefit from more than about 256 characters here.

Bug or negative result? I'm fairly confident the flat context result is real rather than a bug. Re-scoring the saved models myself reproduces every logged validation number, the pattern is the same across all three seeds and both architectures, the library's unit tests pass (28 passed, 3 skipped), and Tiny Shakespeare just doesn't have much structure beyond the current line and the current speaker.

5. Conclusion and limitations

The metric definitions check out to numerical precision, and untrained models score right around ln V, as they should. Once each model is given a fair learning rate, the LSTM and the modern Transformer are tied on this task. The vanilla RNN's big headline gap is mostly a tuning and size issue rather than a gating issue. The larger LSTM overfits. And neither model gains anything measurable from context longer than about 256 characters on this corpus.

Limitations. There are several things I'd want to tighten up with more time.

- I picked learning rates using the validation set, so the "best learning rate" numbers are a little optimistic, and there is no separate test set.
- I only tuned learning rates at length 256 with two seeds, and the context study used the default LSTM learning rate, so the Transformer's 0.024–0.032 lead there would probably shrink with a tuned LSTM.
- The breakdown of the RNN gap depends on the order I applied the changes in, and I only measured gradient reach after training.
- This is a single small corpus at the character level. Longer context might well pay off on larger or more structured text.

Extra credit: a GRU written gate by gate

For the extra credit I wrote a GRU cell by hand, one gate at a time, in the same style as the tutorial's LSTM. It computes a reset gate $r = \sigma(W_{ir}x + W_{hr}h)$, an update gate $z = \sigma(W_{iz}x + W_{hz}h)$, a candidate state $n = \tanh(W_{in}x + r \cdot W_{hn}h)$, and the new state $h' = (1 - z)n + zh$ (biases left out here to keep it readable). I then loaded PyTorch's built-in GRU weights into my version and compared the two in float64. On random networks with one to three layers, and on both of my trained GRU models, the outputs match to within 3.7e-15 and the input gradients to within 7.1e-14. Getting there taught me something: my first guess was that PyTorch stores the gates in the order (z, r, n), and that was off by 0.53. I tried all six possible orders, and only (r, z, n) matches. The code is in the repository linked at the top.

Appendix A. Full results

Every experiment in this report is defined by a sweep file, and every number comes from a run produced by nanolm. The sweep files, per-sweep summary tables, analysis and plotting scripts, figures, training logs, and step-by-step reproduction instructions are all in the repository: https://github.com/nikhil-ghind/cmpe258-hw1-language-models.

Every run

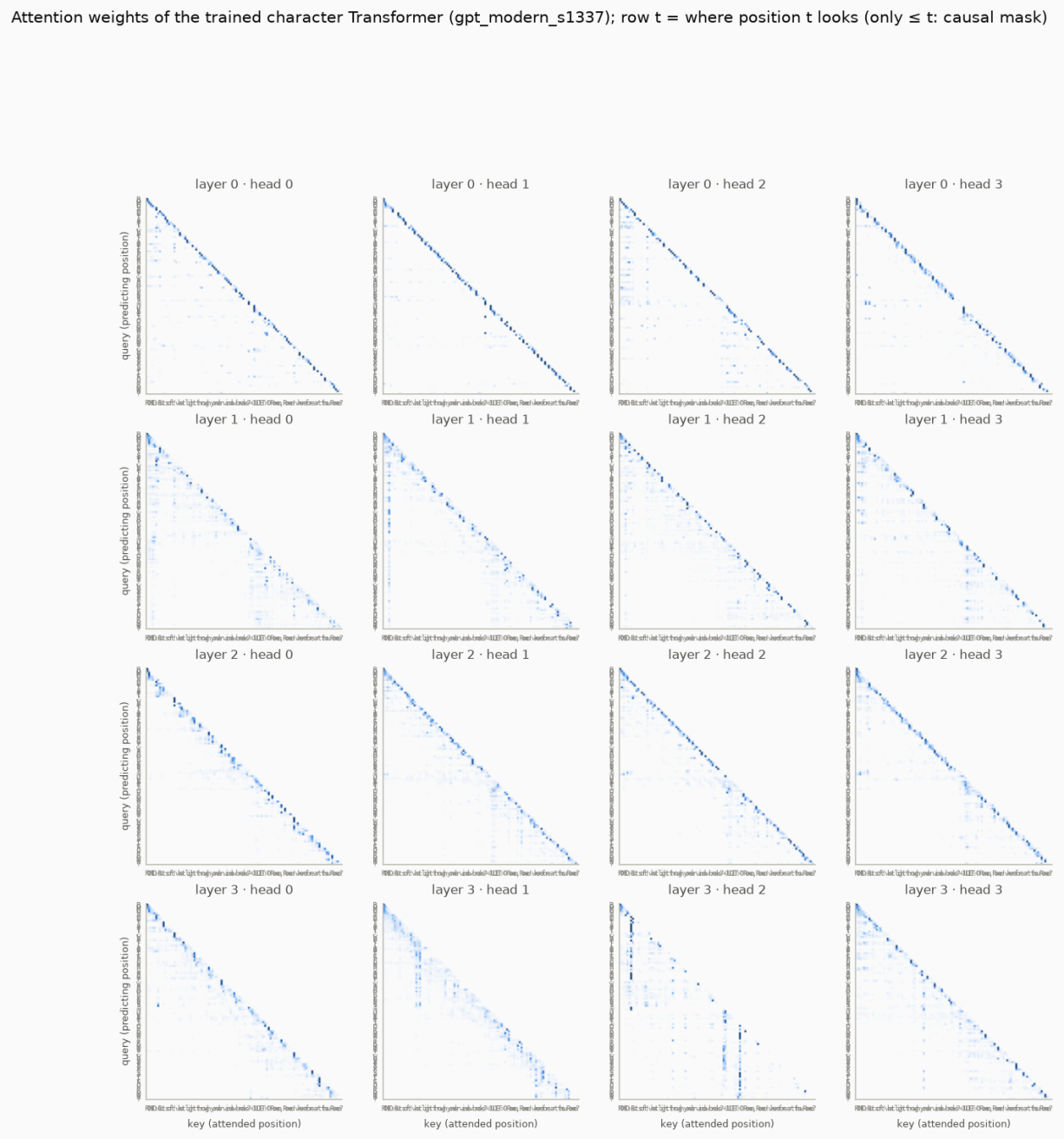
Sweep	Run	Seed	Params (M)	Tokens (M)	Loss	PPL	bpb	Val tokens	Train s
char_models	gpt_modern	1337	3.163	16.384	1.4579	4.297	2.1034	111,360	47
char_models	gpt_modern	42	3.163	16.384	1.4578	4.296	2.1031	111,360	47
char_models	gpt_textbook	1337	3.438	16.384	1.5071	4.513	2.1742	111,360	33
char_models	gpt_textbook	42	3.438	16.384	1.5168	4.558	2.1883	111,360	33
char_models	gru	1337	0.807	16.384	1.4954	4.461	2.1575	111,360	17
char_models	gru	42	0.807	16.384	1.5024	4.493	2.1676	111,360	18
char_models	lstm_512	1337	4.237	16.384	1.5054	4.506	2.1718	111,360	36
char_models	lstm_512	42	4.237	16.384	1.5006	4.484	2.1648	111,360	36
char_models	lstm	1337	1.070	16.384	1.4857	4.418	2.1434	111,360	20
char_models	lstm	42	1.070	16.384	1.4740	4.367	2.1266	111,360	17
char_models	rnn	1337	0.280	16.384	1.6915	5.428	2.4404	111,360	6
char_models	rnn	42	0.280	16.384	1.6910	5.425	2.4397	111,360	6
hw1_rnn_controls	rnn_512_paramsmatch	1337	1.085	16.384	1.5480	4.702	2.2333	111,360	18
hw1_rnn_controls	rnn_512_paramsmatch	42	1.085	16.384	1.5321	4.628	2.2104	111,360	18
hw1_rnn_controls	rnn_lr1e-3	1337	0.280	16.384	2.0182	7.525	2.9116	111,360	6
hw1_rnn_controls	rnn_lr1e-3	42	0.280	16.384	2.0273	7.594	2.9248	111,360	11
hw1_rnn_controls	rnn_lr4e-3	1337	0.280	16.384	1.5654	4.785	2.2585	111,360	6
hw1_rnn_controls	rnn_lr4e-3	42	0.280	16.384	1.5747	4.829	2.2718	111,360	6
hw1_rnn_controls	rnn_lr5e-4	1337	0.280	16.384	2.1011	8.175	3.0312	111,360	11
hw1_rnn_controls	rnn_lr5e-4	42	0.280	16.384	2.0988	8.157	3.0280	111,360	11
hw1_lr_controls	gru_lr1e-3	1337	0.807	16.384	1.6201	5.054	2.3373	111,360	29
hw1_lr_controls	gru_lr1e-3	42	0.807	16.384	1.6139	5.022	2.3283	111,360	30
hw1_lr_controls	gru_lr4e-3	1337	0.807	16.384	1.4573	4.294	2.1025	111,360	28
hw1_lr_controls	gru_lr4e-3	42	0.807	16.384	1.4718	4.357	2.1234	111,360	28
hw1_lr_controls	lstm_lr1e-3	1337	1.070	16.384	1.5829	4.869	2.2837	111,360	32
hw1_lr_controls	lstm_lr1e-3	42	1.070	16.384	1.5837	4.873	2.2848	111,360	32
hw1_lr_controls	lstm_lr4e-3	1337	1.070	16.384	1.4565	4.291	2.1013	111,360	32
hw1_lr_controls	lstm_lr4e-3	42	1.070	16.384	1.4563	4.290	2.1009	111,360	32
hw1_lr_controls	rnn512_lr4e-3	1337	1.085	16.384	1.5017	4.489	2.1664	111,360	34
hw1_lr_controls	rnn512_lr4e-3	42	1.085	16.384	1.4898	4.436	2.1494	111,360	50
hw1_lr_controls	rnn512_lr8e-3	1337	1.085	16.384	1.5444	4.685	2.2281	111,360	34
hw1_lr_controls	rnn512_lr8e-3	42	1.085	16.384	1.5445	4.686	2.2283	111,360	32
hw1_lr_controls	rnn_lr1.6e-2	1337	0.280	16.384	1.6822	5.377	2.4268	111,360	10
hw1_lr_controls	rnn_lr1.6e-2	42	0.280	16.384	1.6845	5.390	2.4302	111,360	10
hw1_lr_controls	rnn_lr8e-3	1337	0.280	16.384	1.5543	4.732	2.2424	111,360	10
hw1_lr_controls	rnn_lr8e-3	42	0.280	16.384	1.5581	4.750	2.2478	111,360	10
hw1_lr_controls2	gpt_modern_lr2e-3	1337	3.163	16.384	1.4590	4.301	2.1048	111,360	131
hw1_lr_controls2	gpt_modern_lr2e-3	42	3.163	16.384	1.4588	4.301	2.1046	111,360	131
hw1_lr_controls2	gpt_modern_lr4e-3	1337	3.163	16.384	1.4720	4.358	2.1237	111,360	131
hw1_lr_controls2	gpt_modern_lr4e-3	42	3.163	16.384	1.4659	4.331	2.1149	111,360	131
hw1_lr_controls2	gpt_modern_lr5e-4	1337	3.163	16.384	1.4733	4.364	2.1256	111,360	129
hw1_lr_controls2	gpt_modern_lr5e-4	42	3.163	16.384	1.4657	4.331	2.1146	111,360	130
hw1_lr_controls2	gru_lr8e-3	1337	0.807	16.384	1.4720	4.358	2.1237	111,360	28
hw1_lr_controls2	gru_lr8e-3	42	0.807	16.384	1.4681	4.341	2.1180	111,360	27
hw1_lr_controls2	lstm_lr8e-3	1337	1.070	16.384	1.4568	4.292	2.1017	111,360	32
hw1_lr_controls2	lstm_lr8e-3	42	1.070	16.384	1.4730	4.362	2.1251	111,360	32
hw1_context	gpt_T128	1337	3.163	16.384	1.4681	4.341	2.1180	111,488	125
hw1_context	gpt_T128	42	3.163	16.384	1.4667	4.335	2.1160	111,488	124
hw1_context	gpt_T128	7	3.163	16.384	1.4772	4.381	2.1312	111,488	124

hw1_context	gpt_T256	1337	3.163	16.384	1.4555	4.287	2.0998	111,360	126
hw1_context	gpt_T256	42	3.163	16.384	1.4623	4.316	2.1097	111,360	127
hw1_context	gpt_T256	7	3.163	16.384	1.4601	4.306	2.1064	111,360	126
hw1_context	gpt_T512	1337	3.163	16.384	1.4535	4.278	2.0970	111,104	129
hw1_context	gpt_T512	42	3.163	16.384	1.4564	4.291	2.1012	111,104	128
hw1_context	gpt_T512	7	3.163	16.384	1.4653	4.329	2.1140	111,104	128
hw1_context	gpt_T64	1337	3.163	16.384	1.5004	4.483	2.1646	111,488	130
hw1_context	gpt_T64	42	3.163	16.384	1.4912	4.442	2.1514	111,488	126
hw1_context	gpt_T64	7	3.163	16.384	1.4977	4.471	2.1607	111,488	125
hw1_context	lstm_T128	1337	1.070	16.384	1.4935	4.453	2.1547	111,488	27
hw1_context	lstm_T128	42	1.070	16.384	1.4912	4.442	2.1513	111,488	27
hw1_context	lstm_T128	7	1.070	16.384	1.4929	4.450	2.1537	111,488	27
hw1_context	lstm_T256	1337	1.070	16.384	1.4856	4.418	2.1433	111,360	31
hw1_context	lstm_T256	42	1.070	16.384	1.4740	4.367	2.1265	111,360	31
hw1_context	lstm_T256	7	1.070	16.384	1.4827	4.405	2.1390	111,360	31
hw1_context	lstm_T512	1337	1.070	16.384	1.4747	4.370	2.1275	111,104	37
hw1_context	lstm_T512	42	1.070	16.384	1.4722	4.359	2.1240	111,104	37
hw1_context	lstm_T512	7	1.070	16.384	1.4773	4.381	2.1314	111,104	40
hw1_context	lstm_T64	1337	1.070	16.384	1.5173	4.560	2.1890	111,488	28
hw1_context	lstm_T64	42	1.070	16.384	1.5184	4.565	2.1905	111,488	28
hw1_context	lstm_T64	7	1.070	16.384	1.5094	4.524	2.1776	111,488	28
hw1_untrained	byte	1337	3.212	0.008	5.4435	231.243	7.8533	111,360	0
hw1_untrained	byte	42	3.212	0.008	5.4406	230.589	7.8492	111,360	0
hw1_untrained	char	1337	3.163	0.008	4.1601	64.078	6.0018	111,360	0
hw1_untrained	char	42	3.163	0.008	4.1760	65.107	6.0247	111,360	0
hw1_untrained	gpt2	1337	16.012	0.008	10.7927	48664.697	5.0353	35,840	0
hw1_untrained	gpt2	42	16.012	0.008	10.7992	48981.354	5.0384	35,840	0
hw1_untrained_lr0	byte	1337	3.212	0.008	5.5118	247.589	7.9518	111,360	0
hw1_untrained_lr0	byte	42	3.212	0.008	5.5089	246.884	7.9477	111,360	0
hw1_untrained_lr0	char	1337	3.163	0.008	4.2166	67.799	6.0832	111,360	0
hw1_untrained_lr0	char	42	3.163	0.008	4.2320	68.854	6.1055	111,360	0
hw1_untrained_lr0	gpt2	1337	16.012	0.008	10.8216	50092.424	5.0488	35,840	1
hw1_untrained_lr0	gpt2	42	16.012	0.008	10.8286	50444.378	5.0521	35,840	1

Appendix B. Attention maps

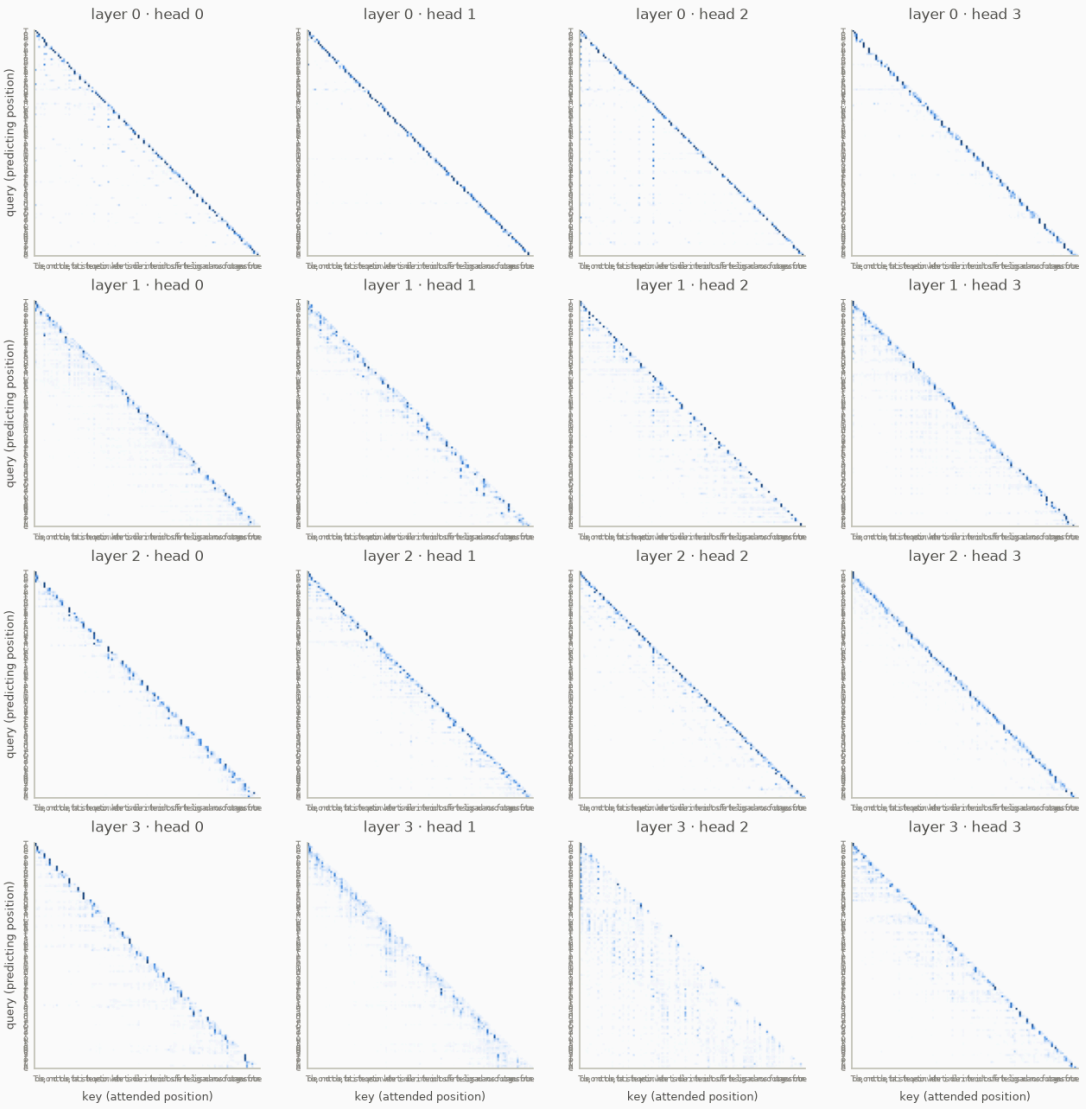
Texts used. Dialogue: "ROMEO: / But soft! what light through yonder window breaks? / / JULIET: / O Romeo, Romeo! wherefore art thou Romeo?". Verse: "To be, or not to be, that is the question: whether tis nobler in the mind to suffer the slings and arrows of outrageous fortune". Modern prose: "The students trained a small language model on their laptop, and then they measured how well it could predict the next letter.".

Dialogue



Verse

Attention weights of the trained character Transformer (gpt_modern_s1337); row t = where position t looks (only ≤ t: causal mask)



Modern prose

Attention weights of the trained character Transformer (gpt_modern_s1337); row t = where position t looks (only ≤ t: causal mask)

